

DISPENZA DELLA LEZIONE

DOM: testo, elementi, classi e attributi

Lettura, modifica e creazione dinamica degli elementi HTML.

CAPITOLO 1

Testo

NOTA

Le proprietà `innerHTML`, `textContent` e `innerText` permettono di leggere o modificare il contenuto di un elemento. `style` permette invece di modificarne lo stile direttamente da JavaScript.

innerHTML

`innerHTML` legge o imposta il contenuto HTML di un elemento, compresi i tag al suo interno. Quando lo imposti con una stringa, il browser la interpreta come HTML e aggiorna il DOM di conseguenza. Va usato con attenzione se il contenuto proviene dall'utente, perché un tag come `<script>` potrebbe essere eseguito, aprendo a vulnerabilità XSS.

textContent

`textContent` legge o imposta il testo puro di un elemento, ignorando qualsiasi tag HTML. Se imposti una stringa che contiene tag, questi vengono mostrati come testo normale e non interpretati dal browser. È la scelta più sicura quando devi aggiornare del testo senza bisogno di markup.

innerText

`innerText` funziona in modo simile a `textContent`, ma tiene conto dello stile CSS applicato alla pagina. Per esempio, il testo di un elemento nascosto con `display: none` non viene incluso nel valore restituito. È più lento di `textContent` perché deve calcolare il rendering visivo prima di rispondere.

style

`style` permette di leggere e modificare gli stili inline di un elemento direttamente da JavaScript. Ogni proprietà CSS viene scritta in notazione camelCase: `background-color` diventa `backgroundColor`, `font-size` diventa `fontSize`, `margin-top` diventa `marginTop`. Gli stili impostati con `style` hanno priorità sulle regole del foglio CSS esterno.

Esempio

```
const elemento = document.querySelector("#elem");

// innerHTML: interpreta i tag
elemento.innerHTML = "<strong>Testo in grassetto</strong>";

// textContent: tratta tutto come testo semplice
elemento.textContent = "<strong>Questo non è grassetto</strong>";

// innerText: legge solo il testo visibile
console.log(elemento.innerText);

// style: modifica lo stile inline
elemento.style.color = "red";
elemento.style.backgroundColor = "#f0f0f0";
elemento.style.fontSize = "18px";
```

CAPITOLO 2

Elementi

NOTA

`createElement()`, `appendChild()`, `insertBefore()` e `remove()` permettono di costruire e modificare la struttura del DOM aggiungendo o eliminando elementi. `parentElement` permette invece di risalire al genitore di un nodo.

parentElement

`parentElement` restituisce l'elemento HTML genitore del nodo su cui viene chiamato. Serve per risalire nella struttura del DOM partendo da un elemento figlio. Se il genitore non è un elemento HTML (per esempio è il documento stesso), restituisce `null`.

createElement()

`createElement()` crea un nuovo elemento HTML e lo restituisce come oggetto, ma non lo inserisce ancora nella pagina. Devi passargli il nome del tag come stringa, ad esempio `"p"`, `"div"` o `"button"`. Dopo averlo creato, puoi impostarne il contenuto, le classi e gli attributi prima di aggiungerlo al DOM.

appendChild()

`appendChild()` aggiunge un elemento come ultimo figlio del nodo su cui viene chiamato. Se l'elemento è già presente nel DOM in un altro punto, viene spostato nella nuova posizione invece di essere duplicato. È il metodo più diretto per inserire un nodo alla fine di un contenitore.

insertBefore()

`insertBefore()` inserisce un nuovo elemento prima di un elemento figlio già esistente. Viene chiamato sul genitore e riceve due argomenti: il nuovo elemento da inserire e l'elemento di riferimento davanti al quale posizionarlo. Se il secondo argomento è `null`, l'elemento viene aggiunto in fondo, come con `appendChild()`.

remove()

`remove()` elimina l'elemento dal DOM chiamandolo direttamente sull'elemento stesso, senza bisogno di riferirsi al genitore. È il metodo più moderno e semplice per rimuovere un nodo dalla pagina.

Esempio

```
const lista = document.querySelector("ul");
const primo = document.querySelector("ul li:first-child");

// Crea un elemento e aggiungilo in fondo
const nuovo = document.createElement("li");
nuovo.textContent = "Ultima voce";
lista.appendChild(nuovo);

// Inserisce prima del primo elemento
const altro = document.createElement("li");
altro.textContent = "Prima voce";
lista.insertBefore(altro, primo);

// Legge il genitore
console.log(nuovo.parentElement);

// Rimuove un elemento
altro.remove();
```

CAPITOLO 3

Classi

NOTA

`classList` è l'oggetto che gestisce le classi CSS di un elemento. Permette di aggiungere, rimuovere e alternare classi senza dover riscrivere l'intero attributo `class`.

`classList.add()`

`classList.add()` aggiunge una o più classi all'elemento. Se una delle classi è già presente, non viene duplicata. Puoi passare più classi come argomenti separati: `classList.add("prima", "seconda")`.

`classList.remove()`

`classList.remove()` rimuove una o più classi dall'elemento. Se la classe non è presente, non genera errori e non modifica nulla. Come per `add()`, puoi passare più classi contemporaneamente.

`classList.toggle()`

`classList.toggle()` aggiunge la classe se non è presente, la rimuove se c'è già. È particolarmente utile per gestire stati alternati come aperto/chiuso, attivo/inattivo, visibile/nascosto. Ogni chiamata inverte lo stato corrente della classe.

Esempio

```
const card = document.querySelector(".card");

// Aggiunge una classe
card.classList.add("attiva");

// Aggiunge più classi insieme
card.classList.add("evidenza", "grande");

// Rimuove una classe
card.classList.remove("evidenza");

// Toggle: aggiunge se assente, rimuove se presente
card.classList.toggle("aperta");
```

CAPITOLO 4

Attributi

NOTA

`setAttribute()`, `getAttribute()`, `removeAttribute()` e `hasAttribute()` permettono di leggere e modificare gli attributi HTML di un elemento, come `href`, `src`, `alt`, `title` o `disabled`.

setAttribute()

`setAttribute()` imposta un attributo sull'elemento con il nome e il valore indicati. Se l'attributo esiste già, ne aggiorna il valore. Se non esiste, lo crea. Riceve due argomenti: il nome dell'attributo e il valore da assegnargli.

getAttribute()

`getAttribute()` restituisce il valore dell'attributo indicato come stringa. Se l'attributo non è presente sull'elemento, restituisce `null`. È utile per leggere attributi come `href`, `src` o attributi personalizzati come `data-id`.

removeAttribute()

`removeAttribute()` rimuove completamente l'attributo dall'elemento. Se l'attributo non è presente, non genera errori. È diverso da impostare un attributo a stringa vuota: in quel caso l'attributo rimane, con `removeAttribute()` scompare del tutto.

hasAttribute()

`hasAttribute()` restituisce `true` se l'attributo è presente sull'elemento, `false` altrimenti. È utile da usare come controllo prima di leggere o rimuovere un attributo, per essere sicuri che esista davvero.

Esempio

```
const immagine = document.querySelector("img");
const link = document.querySelector("a");

// Imposta e Legge un attributo
immagine.setAttribute("alt", "Descrizione immagine");
console.log(immagine.getAttribute("alt"));

// Imposta target su un link
link.setAttribute("target", "_blank");
console.log(link.getAttribute("href"));

// Verifica e rimuove
if (link.hasAttribute("title")) {
    link.removeAttribute("title");
}
```

CAPITOLO 5

Template literal

NOTA

I template literal possono essere combinati con `innerHTML` per costruire blocchi di HTML in modo leggibile, usando variabili e valori dinamici direttamente dentro la stringa.

Come funziona

Un template literal è una stringa delimitata da backtick che permette di inserire espressioni JavaScript con la sintassi `${...}`. Quando lo usi con `innerHTML`, puoi costruire strutture HTML complete includendo valori dinamici, senza dover concatenare pezzi di stringa. Il risultato è codice più leggibile e facile da scrivere, soprattutto quando la struttura HTML da generare è complessa o contiene più elementi.

Vantaggi rispetto a createElement

Con `innerHTML` e template literal puoi scrivere l'intera struttura HTML in poche righe, come se stessi scrivendo direttamente nel file HTML. Questo è molto comodo quando devi generare blocchi con più tag annidati: con `createElement()` avresti bisogno di creare ogni elemento separatamente, impostarne il contenuto, e poi collegarlo al genitore passo dopo passo. Il codice risultante sarebbe molto più lungo.

Svantaggi e attenzioni

Il limite principale di `innerHTML` è la sicurezza: se il contenuto inserito proviene da una fonte esterna o dall'utente, potrebbero essere iniettati tag pericolosi come `<script>`, causando vulnerabilità XSS. Inoltre, ogni volta che imposti `innerHTML` l'intero contenuto del nodo viene distrutto e ricreato, il che può essere inefficiente se hai solo bisogno di aggiornare un singolo elemento figlio. In quei casi, `createElement()` e `appendChild()` sono più precisi e controllati.

Esempio

```
const contenitore = document.querySelector(".contenitore");

const titolo = "JavaScript Base";
const descrizione = "Manipolazione del DOM";
const anno = 2026;

// Con innerHTML e template literal
contenitore.innerHTML = `
  <div class="card">
    <h2>${titolo}</h2>
    <p>${descrizione}</p>
    <small>Anno: ${anno}</small>
  </div>
`;

// Lo stesso risultato con createElement richiederebbe:
const card = document.createElement("div");
card.classList.add("card");

const h2 = document.createElement("h2");
h2.textContent = titolo;

const p = document.createElement("p");
p.textContent = descrizione;

const small = document.createElement("small");
small.textContent = "Anno: " + anno;

card.appendChild(h2);
card.appendChild(p);
card.appendChild(small);
contenitore.appendChild(card);
```